Test 1

Code

Task 1 | English

A non-empty array A consisting of N numbers is given. The array is sorted in non-decreasing order. The *absolute distinct count* of this array is the number of distinct absolute values among the elements of the array.

For example, consider array A such that:

```
A[0] = -5
A[1] = -3
A[2] = -1
A[3] =  0
A[4] =  3
A[5] =  6
```

The absolute distinct count of this array is 5, because there are 5 distinct absolute values among the elements of this array, namely 0, 1, 3, 5 and 6.

Write a function:

```
class Solution { public int solution(int[] A); }
```

Files

- task1
  - solution.java
  - test-input.txt

solution.java — Language: Java 21

```java
class Solution {
    public int solution(int[] A) {
        int left = 0;
        int right = A.length - 1;
        int count = 0;

        while (left <= right) {
            long leftAbs = Math.abs((long) A[left]);
            long rightAbs = Math.abs((long) A[right]);

            long current = Math.max(leftAbs, rightAbs);
            count++;

            // Move left pointer
            while (left <= right && Math.abs((long) A[left]) == current) {
                left++;
            }

            // Move right pointer
            while (left <= right && Math.abs((long) A[right]) == current) {
                right--;
            }
        }

        return count;
    }
}
```

To leave editor use Ctrl + M — Language Version: Java SE 21

Test Output — Run code

Autocomplete is connected | All changes saved — Give Feedback

Output

Test Output — Run code

```
Compilation successful.

Example test:   [-5, -3, -1, 0, 3, 6]
OK

Your code is syntactically correct and works properly on the example test.
Note that the example tests are not part of your score. On submission at least 8 test cases not shown here will assess your solution.
```

Result

Analysis

Detected time complexity: **O(N) or O(N*log(N))**

collapse all	Example tests	
▼ example example test		✓ OK
1. 0.023 s	OK	

collapse all	Correctness tests	
▼ one_element		✓ OK
1. 0.023 s	OK	
2. 0.023 s	OK	
3. 0.023 s	OK	
▼ two_elements		✓ OK
1. 0.023 s	OK	
2. 0.023 s	OK	
3. 0.023 s	OK	
4. 0.023 s	OK	
5. 0.023 s	OK	
6. 0.023 s	OK	
7. 0.023 s	OK	
8. 0.022 s	OK	
9. 0.023 s	OK	
▼ same_elements		✓ OK

▼ same_elements		✓ OK
1. 0.023 s	OK	
2. 0.023 s	OK	
3. 0.022 s	OK	
▼ simple		✓ OK
1. 0.022 s	OK	
▼ simple_no_zero		✓ OK
1. 0.022 s	OK	
▼ simple_no_same		✓ OK
1. 0.022 s	OK	
▼ simple_no_negative		✓ OK
1. 0.022 s	OK	
▼ simple_no_positive		✓ OK
1. 0.022 s	OK	
▼ arith_overlow		✓ OK
1. 0.022 s	OK	
2. 0.022 s	OK	
3. 0.022 s	OK	
▼ medium_chaotic1		✓ OK
1. 0.022 s	OK	
▼ medium_chaotic2		✓ OK
1. 0.022 s	OK	

collapse all	Performance tests	
▼ long_sequence_no_negative		✓ OK
1. 0.069 s	OK	
▼ long_sequence_no_positive		✓ OK
1. 0.062 s	OK	
▼ long_sequence		✓ OK
1. 0.108 s	OK	

Test 2

Code

```java
class Solution {
    public int solution(int M, int[] A) {
        boolean[] seen = new boolean[M + 1];
        int left = 0;
        int count = 0;
        int limit = 1_000_000_000;

        for (int right = 0; right < A.length; right++) {

            while (seen[A[right]]) {
                seen[A[left]] = false;
                left++;
            }

            seen[A[right]] = true;
            count += (right - left + 1);

            if (count >= limit) {
                return limit;
            }
        }

        return count;
    }
}
```

Output

Test Output

Compilation successful.

Example test: (6, [3, 4, 5, 5, 2])
OK

Your code is syntactically correct and works properly on the example test.
Note that the example tests are not part of your score. On submission at least 8 test cases not shown here will assess your solution.

Result

Analysis

Detected time complexity: O(N)

collapse all	Example tests	
▼ example example test		✓ OK
1. 0.024 s	OK	

collapse all	Correctness tests	
▼ single single element		✓ OK
1. 0.024 s	OK	
2. 0.024 s	OK	
3. 0.023 s	OK	
▼ double double elements		✓ OK
1. 0.024 s	OK	
2. 0.023 s	OK	
3. 0.024 s	OK	
▼ simple1 first simple test		✓ OK
1. 0.024 s	OK	
▼ simple2 second simple test		✓ OK
1. 0.023 s	OK	
▼ small_random small random test, length = 100		✓ OK
1. 0.024 s	OK	

collapse all	Performance tests	
▼ medium_random medium random test, length = 500		✓ OK
1. 0.024 s	OK	
▼ large large tests, length = ~100,000		✓ OK
1. 0.087 s	OK	
2. 0.090 s	OK	
▼ large_range large range tests, length = ~100,000		✓ OK
1. 0.126 s	OK	
2. 0.065 s	OK	
▼ large_random large random tests, length = ~100,000		✓ OK
1. 0.131 s	OK	
2. 0.110 s	OK	
▼ extreme_the_same all the same elements, length = ~100,000		✓ OK
1. 0.085 s	OK	
2. 0.141 s	OK	

Test 3

Code

1

Task 1

An array A consisting of N integers is given. A triplet (P, Q, R) is *triangular* if it is possible to build a triangle with sides of lengths A[P], A[Q] and A[R]. In other words, triplet (P, Q, R) is triangular if 0 ≤ P < Q < R < N and:

- A[P] + A[Q] > A[R],
- A[Q] + A[R] > A[P],
- A[R] + A[P] > A[Q].

For example, consider array A such that:

```
A[0] = 10    A[1] = 2    A[2] = 5
A[3] = 1     A[4] = 8    A[5] = 12
```

There are four triangular triplets that can be constructed from elements of this array, namely (0, 2, 4), (0, 2, 5), (0, 4, 5), and (2, 4, 5).

Write a function:

```
class Solution { public int solution(int[] A); }
```

Files

task1

solution.java

test-input.txt

solution.java

Language: Java 21

```java
import java.util.Arrays;

class Solution {
    public int solution(int[] A) {
        int N = A.length;
        int count = 0;

        Arrays.sort(A);

        for (int i = 0; i < N - 2; i++) {
            int k = i + 2;

            for (int j = i + 1; j < N - 1; j++) {

                while (k < N && (long)A[i] + A[j] > A[k]) {
                    k++;
                }

                count += (k - j - 1);
            }
        }

        return count;
    }
}
```

To leave editor use Ctrl + M

Language Version: Java SE 21

Test Output

Run code

Autocomplete is connected | All changes saved

Give Feedback

Output

Test Output

Run code

```
Compilation successful.

Example test:   [10, 2, 5, 1, 8, 12]
OK

Your code is syntactically correct and works properly on the example test.
Note that the example tests are not part of your score. On submission at least 8 test cases not shown here will assess your solution.
```

Result

Analysis

Detected time complexity: **O(N**2)**

collapse all	Example tests	
▼ example example, positive answer, length=6		✓ OK
1. 0.023 s	OK	

collapse all	Correctness tests	
▼ extreme_empty empty sequence + [5,3,3]		✓ OK
1. 0.023 s	OK	
2. 0.023 s	OK	
▼ extreme_single 1-element sequence + [5,3,3]		✓ OK
1. 0.023 s	OK	
2. 0.023 s	OK	
▼ extreme_two_elems 2-element sequence + [5,3,3]		✓ OK
1. 0.024 s	OK	
2. 0.024 s	OK	
▼ extreme_arith_overflow overflow test, 3 MAXINTs + [5,3,3]		✓ OK
1. 0.024 s	OK	
2. 0.024 s	OK	

▼ simple		✓ OK
1. 0.024 s	OK	
▼ medium1 chaotic sequence of values from [1..100K], length=30		✓ OK
1. 0.023 s	OK	
▼ medium2 chaotic sequence of values from [1..1K], length=50		✓ OK
1. 0.023 s	OK	
collapse all	Performance tests	
▼ large chaotic sequence with values from [1..10], length=200		✓ OK
1. 0.024 s	OK	
▼ large2 1 followed by an ascending sequence of ~1K elements from [1..2K]		✓ OK
1. 0.047 s	OK	
▼ large_random chaotic sequence of values from [1..1M], length=1K		✓ OK
1. 0.048 s	OK	
▼ large_the_same sequence of the same value value		✓ OK
1. 0.042 s	OK	
2. 0.043 s	OK	

Test 4

Code

Task 1

Let A be a non-empty array consisting of N integers.

The *abs sum of two* for a pair of indices (P, Q) is the absolute value |A[P] + A[Q]|, for 0 ≤ P ≤ Q < N.

For example, the following array A:

```
A[0] =  1
A[1] =  4
A[2] = -3
```

has pairs of indices (0, 0), (0, 1), (0, 2), (1, 1), (1, 2), (2, 2).

The abs sum of two for the pair (0, 0) is A[0] + A[0] = |1 + 1| = 2.

The abs sum of two for the pair (0, 1) is A[0] + A[1] = |1 + 4| = 5.

The abs sum of two for the pair (0, 2) is A[0] + A[2] = |1 + (−3)| = 2.

The abs sum of two for the pair (1, 1) is A[1]

Files

task1
solution.java
test-input.txt

solution.java

Language: Java 21

```java
import java.util.Arrays;

class Solution {
    public int solution(int[] A) {
        Arrays.sort(A);

        int left = 0;
        int right = A.length - 1;
        int min = Integer.MAX_VALUE;

        while (left <= right) {
            int sum = A[left] + A[right];
            int absSum = Math.abs(sum);

            if (absSum < min) {
                min = absSum;
            }

            if (sum < 0) {
                left++;
            } else if (sum > 0) {
                right--;
            } else {
                return 0; // best possible
            }
        }
```

To leave editor use Ctrl + M

Language Version: Java SE 21

Test Output

Run code

Autocomplete is connected | All changes saved

Give Feedback

Output

Test Output

Run code

```
Compilation successful.

Example test:   [1, 4, -3]
OK

Example test:   [-8, 4, 5, -10, 3]
OK

Your code is syntactically correct and works properly on the example test.
Note that the example tests are not part of your score. On submission at least 8 test cases not shown here will assess your solution.
```

Result

Analysis

Detected time complexity: **O(N * log(N))**

collapse all	Example tests	
▼ example1 first example		✓ OK
1. 0.024 s	OK	
▼ example2 second example		✓ OK
1. 0.024 s	OK	

collapse all	Correctness tests	
▼ extreme_single sequences of 1 elements		✓ OK
1. 0.024 s	OK	
2. 0.025 s	OK	
3. 0.024 s	OK	
▼ extreme_double sequences of 2 elements		✓ OK
1. 0.023 s	OK	
2. 0.023 s	OK	
3. 0.025 s	OK	
▼ positive_small only positive numbers		✓ OK
1. 0.024 s	OK	
▼ negative_small only negative numbers		✓ OK
1. 0.024 s	OK	

collapse all	Performance tests	
▼ random_small random sequence, length = ~1000		✓ OK
1. 0.026 s	OK	
▼ random_medium random sequence, length = ~10,000		✓ OK
1. 0.044 s	OK	
▼ arithmetic_medium arithemtic sequence, length = ~10,000		✓ OK
1. 0.076 s	OK	
▼ random_large random sequence, length = ~100,000		✓ OK
1. 0.224 s	OK	
▼ extreme_large sequence of MAX_INT, length = ~100,000		✓ OK
1. 0.187 s	OK	
▼ arithmetic_large arithmetic sequence, length = ~100,000		✓ OK
1. 0.219 s	OK	
▼ constant_distance constant distance between all elements, length = 100,000		✓ OK
1. 0.197 s	OK	